Example - Creating a ROS2 Waypoint Tracking Controller

Overview

This example demonstrates how to create a custom ROS2 package to interact with the Panisim simulator. We'll build a Guidance, Navigation, and Control (GNC) package that implements a waypoint tracking PID controller for marine vessels. This example covers:

1. Creating a ROS2 package structure
2. Implementing core controller logic
3. Setting up ROS2 nodes to interface with the simulator
4. Building and launching the complete system

> **Tip**
>
> All the below codes and folders are already implemented in the Repository. This example is only for the reference and to show how you can configure your own ROS2 package to interact with the Panisim simulator.

Creating the ROS2 Package

1. Package Structure

First, let's create a new ROS2 package called `gnc` in the `ros2_ws/src` directory:

```
cd ros2_ws/src
ros2 pkg create --build-type ament_python gnc --dependencies rclpy nav_msgs geometry_msgs int
```

This creates a basic package structure:

```
gnc/
   gnc/
       __init__.py
   package.xml
   setup.py
   setup.cfg
```

2. Update Package Dependencies

Edit `package.xml` to include all required dependencies:

```xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://w
<package format="3">
  <name>gnc</name>
  <version>0.0.1</version>
  <description>Guidance, Navigation and Control package for Panisim</description>
  <maintainer email="example@example.com">user</maintainer>
  <license>Apache License 2.0</license>

  <depend>rclpy</depend>
  <depend>nav_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>std_msgs</depend>
  <depend>interfaces</depend>
  <depend>mav_simulator</depend>

  <exec_depend>ros2launch</exec_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

3. Configure Setup Scripts

Update `setup.py` to include our entry points and specify dependencies:

```python
from setuptools import setup
import os
from glob import glob
```

```
package_name = 'gnc'

setup(
    name=package_name,
    version='0.0.1',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
        (os.path.join('share', package_name, 'web'), glob('web/**/*', recursive=True)),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='user',
    maintainer_email='example@example.com',
    description='Guidance, Navigation and Control package for Panisim',
    license='Apache License 2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'gc = gnc.guidance_control:main',
            'nav = gnc.navigation:main',
        ],
    },
)
```

4. Create Directory Structure

Set up the complete directory structure:

```
mkdir -p gnc/launch
mkdir -p gnc/web
```

Implementing the Controller Logic

1. Control Module

First, create `module_control.py` in the gnc package to implement the PID control logic:

```python
# ros2_ws/src/gnc/gnc/module_control.py
import numpy as np

"""
Rudder control module for vessel simulation.

This module provides rudder control strategies for vessel steering with PID controllers.
"""

def ssa(ang, deg=False):
    """
    Smallest signed angle that lies between -pi and pi.
    If deg is True, the angle is assumed to be in degrees and is converted to radians.
    If deg is False, the angle is assumed to be in radians.
    """
    if deg:
        ang = (ang + 180) % (360.0) - 180.0
    else:
        ang = (ang + np.pi) % (2 * np.pi) - np.pi
    return ang

def clip(val, min_val, max_val):
    """
    Clip a value to a range.
    """
    return max(min_val, min(val, max_val))

def pid_control(t, state, waypoints, waypoint_idx, ye_int=0.0):
    """
    Implement a PID control strategy to follow the waypoints.

    Args:
        t (float): Current simulation time [s]
        state (ndarray): Current vessel state vector
        waypoints (ndarray): Waypoints array
        waypoint_idx (int): Current waypoint index
```

```
        ye_int (float): Integral term of cross-track error

    Returns:
        float: Commanded rudder angle in radians
        float: Cross-track error
        int: Next waypoint index
    """

    # Check if we have reached the last waypoint
    if waypoint_idx == len(waypoints):
        return 0.0, 0.0, waypoint_idx

    # Current state
    u, v, r = state[3], state[4], state[5]
    x, y, psi = state[6], state[7], state[11]

    # Current and previous waypoints
    wp_xn, wp_yn, _ = waypoints[waypoint_idx]
    wp_xn1, wp_yn1, _ = waypoints[waypoint_idx - 1]

    # Calculate path line equation: ax + by + c = 0
    a = -(wp_yn1 - wp_yn)
    b = (wp_xn1 - wp_xn)
    c = -wp_yn1*b-a*wp_xn1

    # Path direction unit vector
    wp_unit_vec = np.array([wp_xn - wp_xn1, wp_yn - wp_yn1, 0.0])
    wp_unit_vec = wp_unit_vec / np.linalg.norm(wp_unit_vec)

    # Calculate cross-track error
    ye = -(a*x + b*y + c)/np.sqrt(a**2 + b**2)

    # Path direction angle
    pi_p = np.angle(wp_unit_vec[0] + 1j * wp_unit_vec[1])

    # Outer loop PID gains
    Kpo = 0.6  # Proportional gain
    Kio = 0.05 # Integral gain

    # Desired heading (outer loop control)
    psid = pi_p + Kpo*(-ye) + Kio*(-ye_int)
```

```
    # Inner loop PID gains
    Kpi = 0.7  # Proportional gain
    Kdi = 0.5  # Derivative gain

    # Rudder command (inner loop control)
    delta_c = Kpi*ssa(psid - psi) + Kdi*(0 - r)

    # Clip the rudder angle
    delta_c = clip(delta_c, -35*np.pi/180, 35*np.pi/180)

    # Distance to waypoint
    wp_dist = np.linalg.norm(np.array([x - wp_xn, y - wp_yn, 0.0]))
    if wp_dist < 0.5:  # 0.5m threshold
        waypoint_idx += 1

    # Return negative delta_c (due to sign convention of rudder angle)
    return -delta_c, ye, waypoint_idx
```

2. Guidance Control Node Class

Next, create `class_guidance_control.py` to implement the ROS2 node class:

```
# ros2_ws/src/gnc/gnc/class_guidance_control.py
from rclpy.node import Node
from nav_msgs.msg import Odometry
from interfaces.msg import Actuator
from geometry_msgs.msg import PoseArray, Pose
from mav_simulator.module_kinematics import quat_to_eul
import gnc.module_control as con
import numpy as np

class GuidanceControl(Node):
    def __init__(self, vessel):
        super().__init__('guidance_controller')

        self.vessel = vessel

        # Configure topics based on vessel name and ID
        self.odom_topic = f'{self.vessel.vessel_name}_{self.vessel.vessel_id:02d}/odometry_s
        self.actuator_topic = f'{self.vessel.vessel_name}_{self.vessel.vessel_id:02d}/actuat
        self.waypoints_topic = f'{self.vessel.vessel_name}_{self.vessel.vessel_id:02d}/waypo
```

```
        # Create subscriber for odometry
        self.odom_sub = self.create_subscription(
            Odometry,
            self.odom_topic,
            self.odom_callback,
            10)

        # Create publisher for rudder command
        self.actuator_pub = self.create_publisher(
            Actuator,
            self.actuator_topic,
            10)

        # Create publisher for waypoints
        self.waypoints_pub = self.create_publisher(
            PoseArray,
            self.waypoints_topic,
            10)

        # Set up timer to publish waypoints
        self.timer = self.create_timer(1.0, self.publish_waypoints)

        # Read waypoints from vessel configuration
        self.waypoints = np.array(self.vessel.vessel_config['guidance']['waypoints'])
        self.waypoints_type = self.vessel.vessel_config['guidance']['waypoints_type']
        self.waypoint_idx = 1

        self.waypoints_data = {
            'waypoints': self.waypoints,
            'waypoints_type': self.waypoints_type
        }

        # Initialize time
        self.start_time = self.get_clock().now()

        # Initialize integrator
        self.ye_int = 0.0
        self.te_int = 0.0

        # Select control mode
        self.control_mode = con.pid_control
```

```
        self.get_logger().info('Guidance Controller initialized')

    def publish_waypoints(self):
        """Publish waypoints for visualization"""
        if not self.waypoints_data:
            return

        pose_array = PoseArray()
        pose_array.header.stamp = self.get_clock().now().to_msg()
        pose_array.header.frame_id = "map"

        for waypoint in self.waypoints_data.get('waypoints', []):
            pose = Pose()
            pose.position.x = float(waypoint[0])
            pose.position.y = float(waypoint[1])
            pose.position.z = float(waypoint[2])
            pose_array.poses.append(pose)

        self.waypoints_pub.publish(pose_array)

    def odom_callback(self, msg):
        """Process odometry data and calculate control commands"""
        # Get current time in seconds
        current_time = self.get_clock().now()
        t = (current_time - self.start_time).nanoseconds / 1e9

        # Extract orientation quaternion
        quat = np.array([
            msg.pose.pose.orientation.w,
            msg.pose.pose.orientation.x,
            msg.pose.pose.orientation.y,
            msg.pose.pose.orientation.z
        ])

        # Convert quaternion to Euler angles
        eul = quat_to_eul(quat, order='ZYX')

        # Create state vector from odometry
        state = np.array([
            msg.twist.twist.linear.x,
            msg.twist.twist.linear.y,
            msg.twist.twist.linear.z,
```

```
            msg.twist.twist.angular.x,
            msg.twist.twist.angular.y,
            msg.twist.twist.angular.z,
            msg.pose.pose.position.x,
            msg.pose.pose.position.y,
            msg.pose.pose.position.z,
            eul[0],
            eul[1],
            eul[2],
            0.0 # rudder angle
        ])

        # Calculate control command
        rudder_cmd, ye, wp_indx = self.control_mode(t, state, self.waypoints, self.waypoint_i

        # Handle waypoint transitions
        if wp_indx != self.waypoint_idx:
            self.get_logger().info(f'Waypoint {self.waypoint_idx} reached')
            self.waypoint_idx = wp_indx
            self.ye_int = 0.0  # Reset integral term

            # Loop back to first waypoint when all are visited
            if self.waypoint_idx >= len(self.waypoints):
                self.get_logger().info('***All waypoints reached***')
                self.waypoint_idx = 1

        # Calculate time difference for integration
        dt = t - self.te_int
        if dt > 0:  # Avoid division by zero or negative time
            # Update the integral term
            self.ye_int += ye * dt
            self.te_int = t

        # Publish actuator command
        cmd_msg = Actuator()
        cmd_msg.actuator_values = [float(rudder_cmd * 180 / np.pi)]  # Convert to degrees
        cmd_msg.actuator_names = ['cs_1']  # Control surface 1
        cmd_msg.covariance = [0.0]
        self.actuator_pub.publish(cmd_msg)
```

3. Main Entry Point

Create `guidance_control.py` to serve as the entry point for the ROS2 node:

```
# ros2_ws/src/gnc/gnc/guidance_control.py
import rclpy
from gnc.class_guidance_control import GuidanceControl
from mav_simulator.class_world import World
from rclpy.executors import MultiThreadedExecutor

def main():
    # Initialize ROS2
    rclpy.init()

    # Load world and vessels from configuration
    world = World('/workspaces/mavlab/inputs/simulation_input.yml')
    vessels = world.vessels

    # Create multi-threaded executor
    executor = MultiThreadedExecutor()

    # Initialize guidance controllers for all vessels
    gcs = []
    for vessel in vessels:
        gcs.append(GuidanceControl(vessel))
        executor.add_node(gcs[-1])

    try:
        # Start spinning the nodes
        executor.spin()

    finally:
        # Clean up on shutdown
        executor.shutdown()

        for gc in gcs:
            gc.destroy_node()

        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4. Create a Navigation Node Placeholder

Add a simple placeholder for the navigation node to satisfy the launch file requirements:

```
# ros2_ws/src/gnc/gnc/navigation.py
import rclpy
from rclpy.node import Node

class Navigation(Node):
    def __init__(self):
        super().__init__('navigation')
        self.get_logger().info('Navigation node started')

def main():
    rclpy.init()
    node = Navigation()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Creating the Launch File

Create a launch file to start all components of the system:

```
# ros2_ws/src/gnc/launch/gnc_launch.py
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch import LaunchDescription
from launch.actions import ExecuteProcess, IncludeLaunchDescription
from launch.launch_description_sources import AnyLaunchDescriptionSource
from launch_ros.substitutions import FindPackageShare
from launch.substitutions import PathJoinSubstitution
from launch_ros.actions import Node
from mav_simulator.class_world import World
from launch.substitutions import LaunchConfiguration
from launch.actions import GroupAction
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import PushRosNamespace
```

```
def generate_launch_description():

    rosbridge_launch = PathJoinSubstitution([
        FindPackageShare('rosbridge_server'),
        'launch',
        'rosbridge_websocket_launch.xml'
    ])

    # Path to the web directory in gnc package
    web_dir = PathJoinSubstitution([
        FindPackageShare('gnc'),
        'web'
    ])

    web_build_dir = '/workspaces/mavlab/ros2_ws/src/mav_simulator/web'

    return LaunchDescription([
        Node(
            package='mav_simulator',  # Replace with your package name
            executable='simulate',  # Replace with your script name
            name='mavsim',
            output='screen'
        ),
        Node(
            package='gnc',
            executable='nav',
            name='nav',
            output='screen'
        ),
        Node(
            package='gnc',
            executable='gc',
            name='gc',
            output='screen'
        ),
        # Start the HTTP server to serve the web interface
        ExecuteProcess(
            cmd=['python3', '-m', 'http.server', '8000', '--directory', web_build_dir],
            name='http_server',
            output='screen'
        ),
        # Start the rosbridge server
```

```
        IncludeLaunchDescription(
            AnyLaunchDescriptionSource(rosbridge_launch)
        )
    ])
```

Building and Launching the Package

With all the files in place, you can build and launch your custom ROS2 package:

1. Build the Package

```
cd /workspaces/mavlab/ros2_ws
colcon build --packages-select gnc
source install/setup.bash
```

2. Launch the System

```
ros2 launch gnc gnc_launch.py
```

This will start: 1. The Panisim simulator 2. The navigation node 3. The guidance control node with waypoint tracking

Configuring Waypoints

Waypoints for vessel navigation are defined in the `guidance.yaml` input file:

```
waypoints_type: XYZ
waypoints:
- [0.0, 0.0, 0.0]     # Starting point
- [10.0, 0.0, 0.0]    # First waypoint
- [10.0, 10.0, 0.0]   # Second waypoint
- [0.0, 10.0, 0.0]    # Third waypoint
- [0.0, 0.0, 0.0]     # Return to start
```

Monitoring System Performance

Once the system is running, you can monitor various aspects of the waypoint tracking:

```
# View odometry data
ros2 topic echo /<vessel_name>_<id>/odometry_sim

# View actuator commands
ros2 topic echo /<vessel_name>_<id>/actuator_cmd

# View waypoints
ros2 topic echo /<vessel_name>_<id>/waypoints
```

You can also visualize the vessel's movement in the web interface at `http://localhost:8000`.